

COMS BC1016

Introduction to Computational Thinking and Data Science

Lecture 6: Histograms and Functions

Sept 30, 2025

Copyright © 2026 Barnard College

September 28, 2026 1

Announcements

- HW 1 is due on Wednesday
- HW 2 has been assigned
 - Due next week Wednesday
- Starting last week, we started having in-class participation activities
 - These cannot be made up, but I plan to have enough throughout the semester that it's okay to miss some

Last Lecture's Survey

- Last class we took a survey.
- Today we will take some time to explore around and answer some questions
- You may work in small in small groups!
- We will take time at the end to go over the answers

Are you a Barnard student or Columbia student?	What is your major?	Are you left-handed or right-handed?	Roughly how many hours of sleep do you get on a weeknight during the semester?	Roughly how many hours of sleep do you get on a weekend during the semester?	On any given day, roughly how many browser tabs do you have open?	Did you answer honestly to all of the questions?
Barnard	Economics or Economics-Statistics	Right-handed	7.5	9	20	Yes
Barnard	Cognitive Science	Right-handed	7.5	8	10	Yes
Barnard	Economics or Economics-Statistics	Left-handed	7	8	47	No

Group Activity

1. How many respondents said they did not answer honestly?

(Hint: `.select(. .)` can be used to get a column by its label or index, and `.group(column_label)` can be used to count the number of rows of each category in a column)

2. How many Undeclared students are Right-handed?

(Hint: `.where(column_label, predicate)` can be used to filter individuals that meet a criteria)

3. Which would be the most appropriate to visualize the relationship between number of hours of sleep on a weeknight and number of browser tabs open?

4. Based on Vergil, we have 2 econ majors in this class. Do you think the number of students who actually lied matches the number that said they lied? Why or why not?

Last Time: Histograms

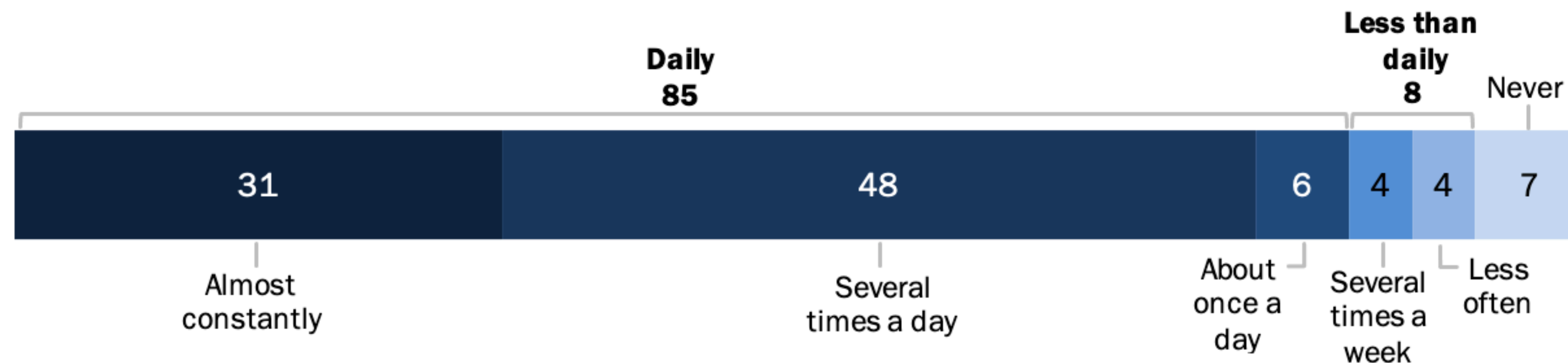
Last Time: Distributions

Each individual has exactly one category

- Percents add to 100

More than eight-in-ten U.S. adults go online at least daily

% of U.S. adults who say they go online ...



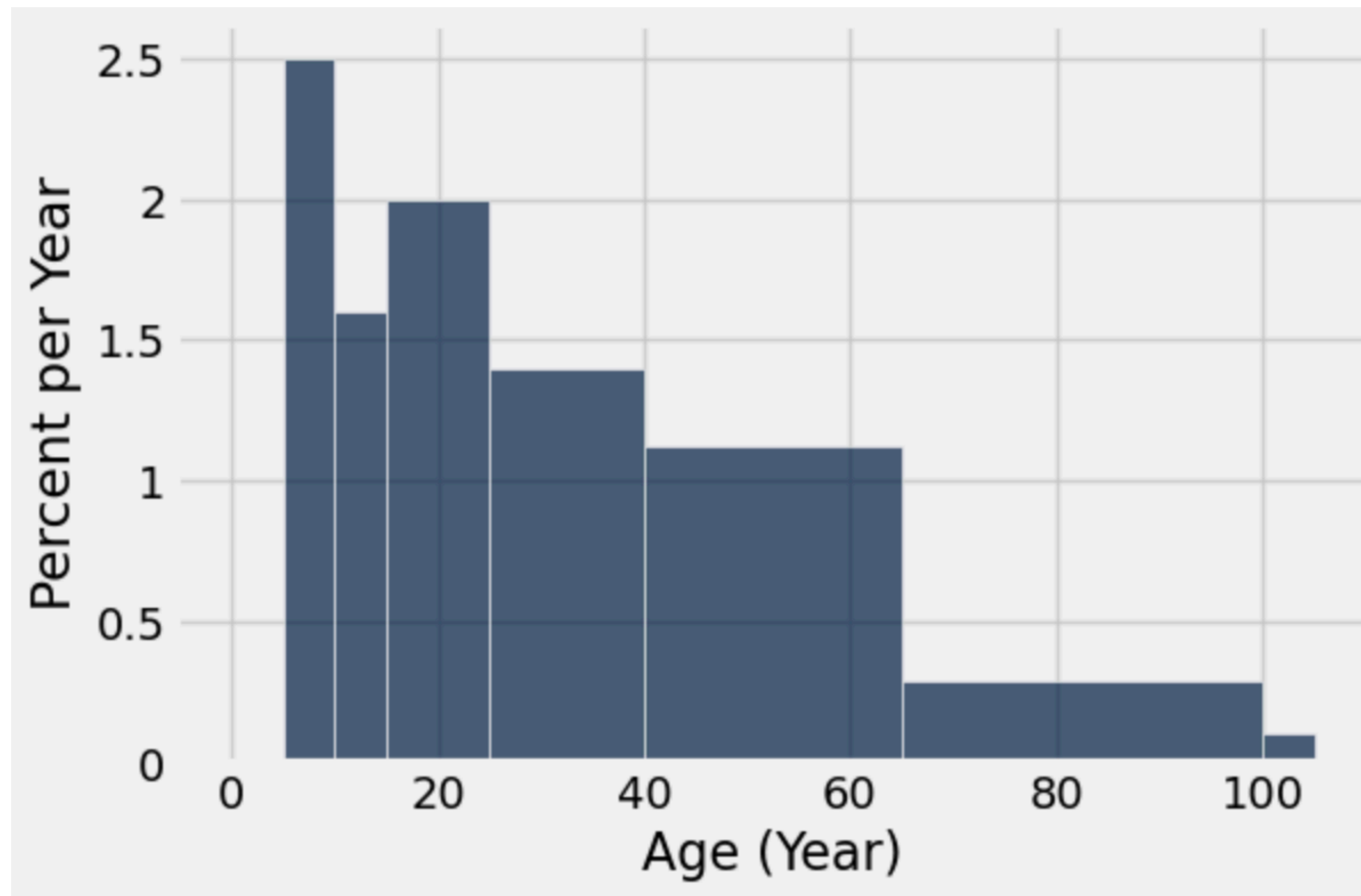
Note: Respondents who did not give an answer are not shown.

Source: Survey of U.S. adults conducted Jan. 25-Feb. 8, 2021.

PEW RESEARCH CENTER

Last Time: Plotting Numerical Distributions

- **Histograms** plot the distribution of a numerical variable using **bins**!

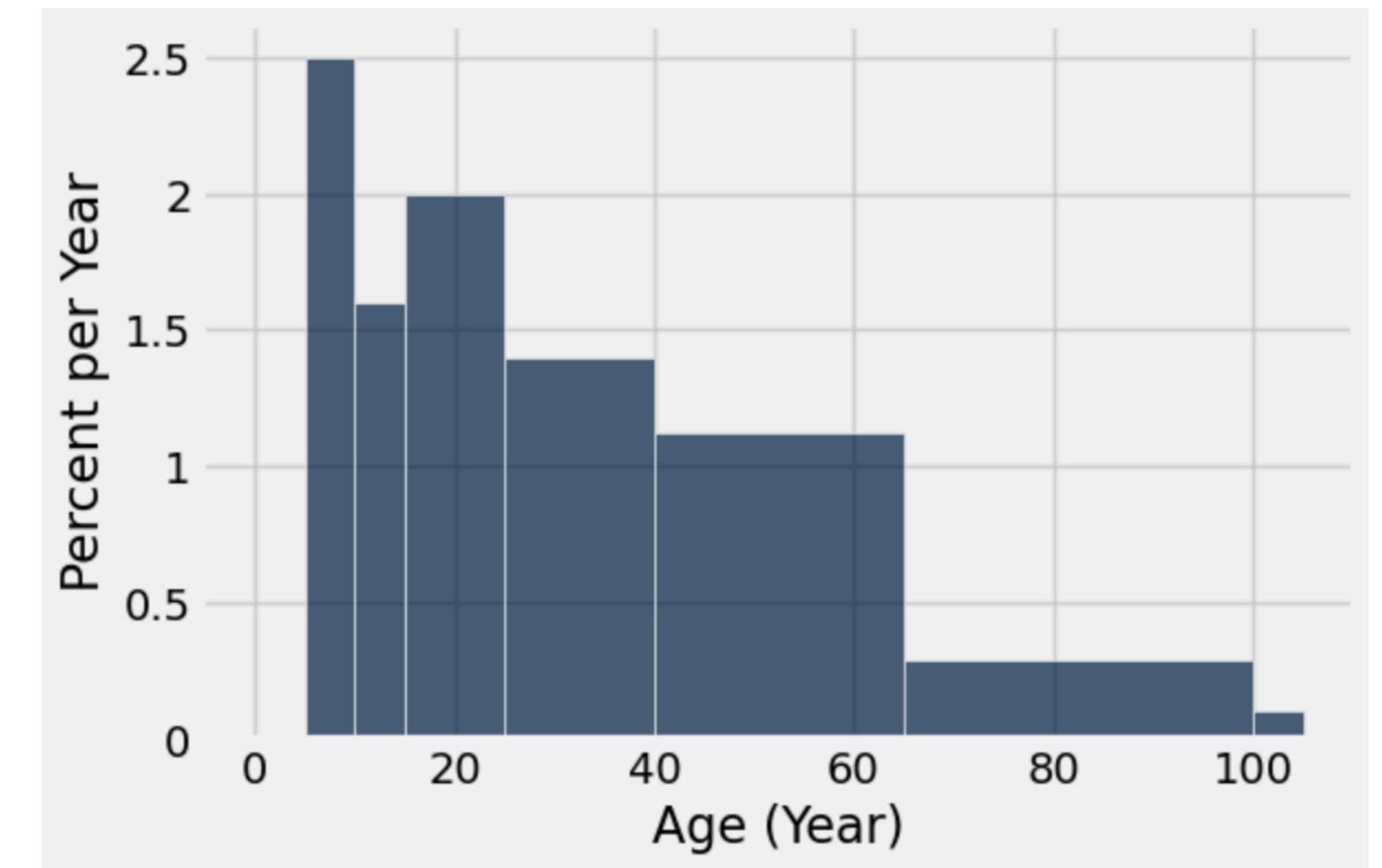


Binning: counting the number of values that lie within a range.

Defined by the lower bound (inclusive).
Upper bound is the lower bound for the next bin

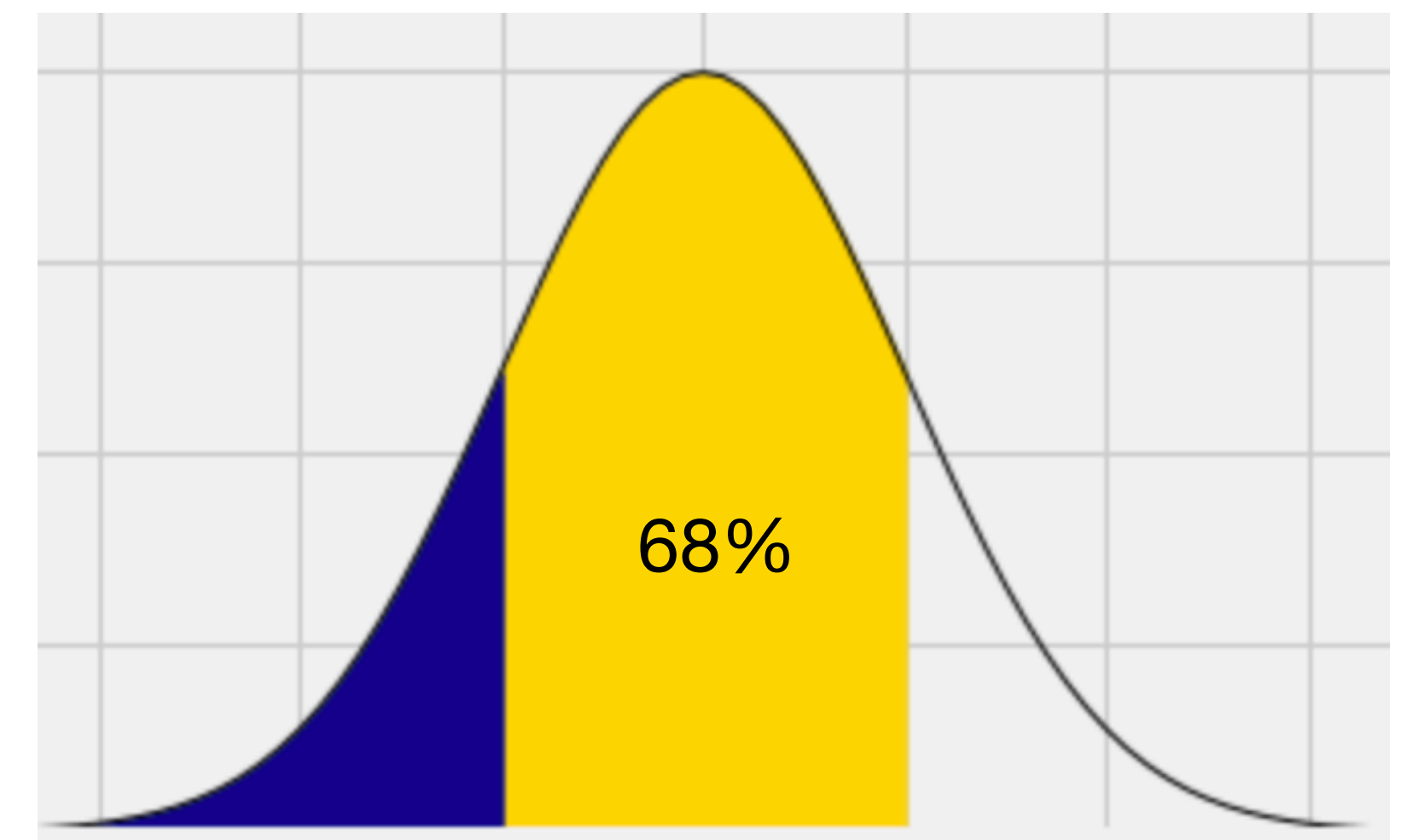
Last Time: Histograms

- One bar corresponds to each bin
- Uses the area principle:
 - The **area** of each bar is the **percent** of individuals in the corresponding bin



Later in the course, we will approximate histograms with smooth curves.

Areas will still represent percents.



Histograms (Continued)

hist

Create a histogram of numerical values in column `c` with 10 equal bins:

- `tbl.hist(c)`

Create a histogram with `u` as the x-axis:

- `tbl.hist(c, unit=u)`

`unit` is an (optional)
argument that changes the
axis labels

Create a histogram with specified bins:

- `tbl.hist(c, bins=np.arange(start, end, step))`

bins can be used
with any array to specify
the histogram bins

Create a histogram with x-axis `u` and specified bins:

- `tbl.hist(c, unit=u, bins=np.arange(start, end, step))`

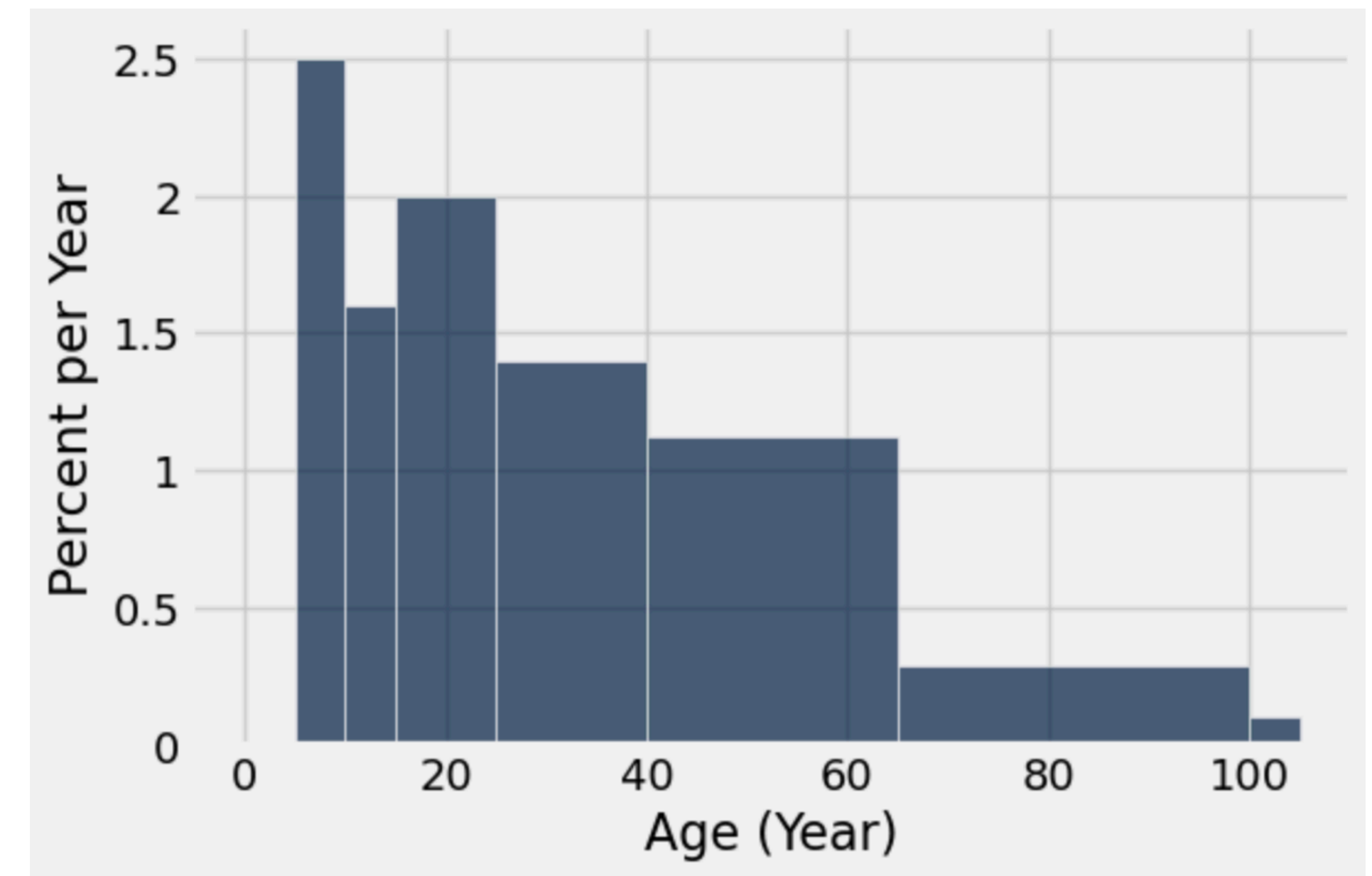
Histograms

By default, `hist` uses a scale that ensures the area of the chart sums to 100%

The **area** of each bar is a **percentage** of the whole

The **horizontal axis** is a number line (e.g., years), and bin sizes don't need to be equal

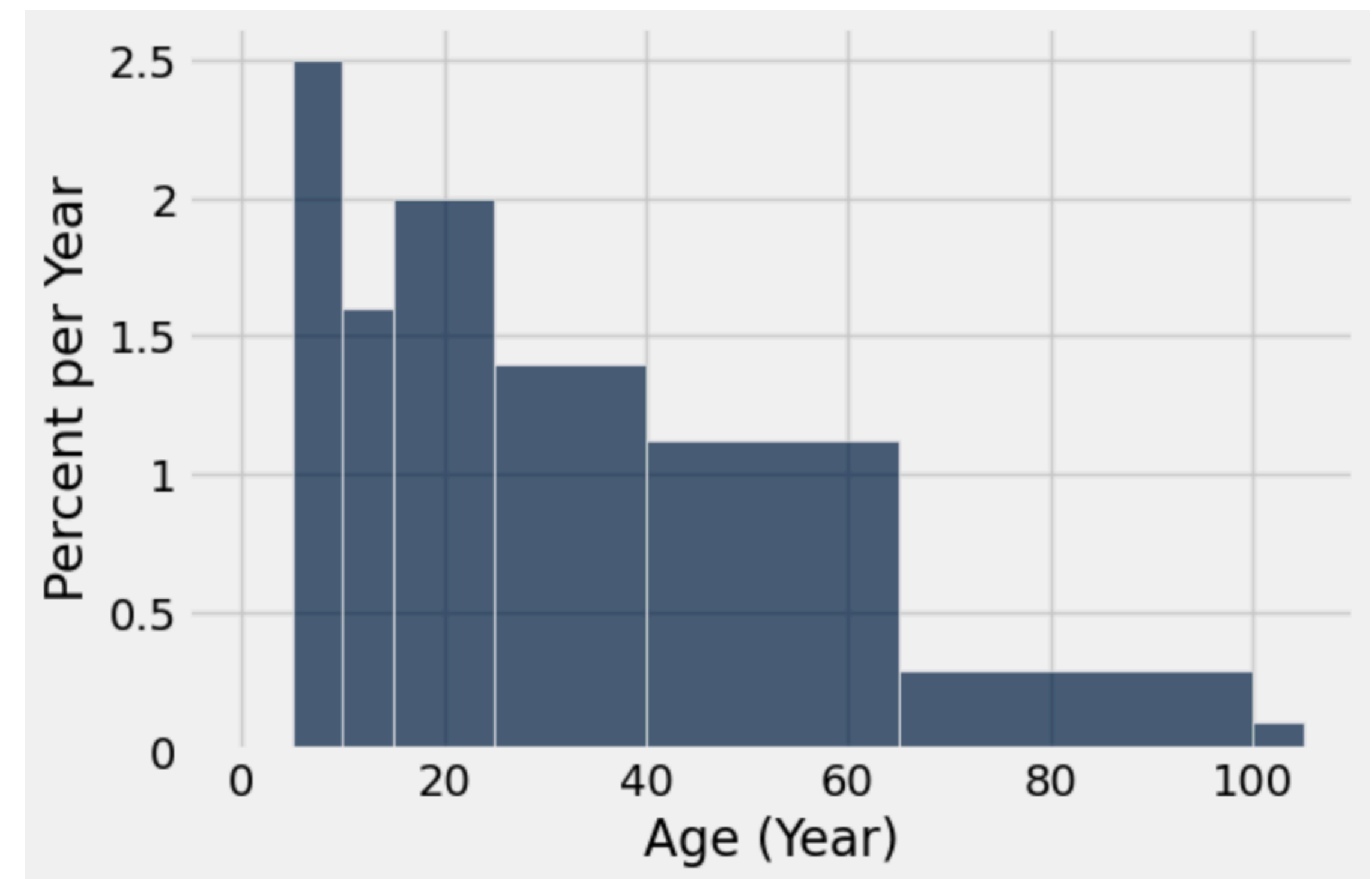
The **vertical axis** is a rate (e.g., percent/year), measuring the density



Height Measures Density

- The height measures the percent of data in the bin *relative to the amount of space in the bin*
- Measures crowdedness or **density**
- Units: percent per x-axis unit

$$\text{height of bar} = \frac{\text{percent of entries in bin}}{\text{width of bin}}$$



How to Calculate Height

The [40, 65) bin contains 59 out of 200 movies

$$\text{height of bar} = \frac{\text{percent of entries in bin}}{\text{width of bin}}$$

How to Calculate Height

The [40, 65) bin contains 59 out of 200 movies

- What percentage is 59 out of 200?

$$59/200 = 29.5\%$$

- How wide is the [40, 65) bin?

$$65-40 = 25 \text{ years wide}$$

$$\text{height of bar} = \frac{\text{percent of entries in bin}}{\text{width of bin}}$$

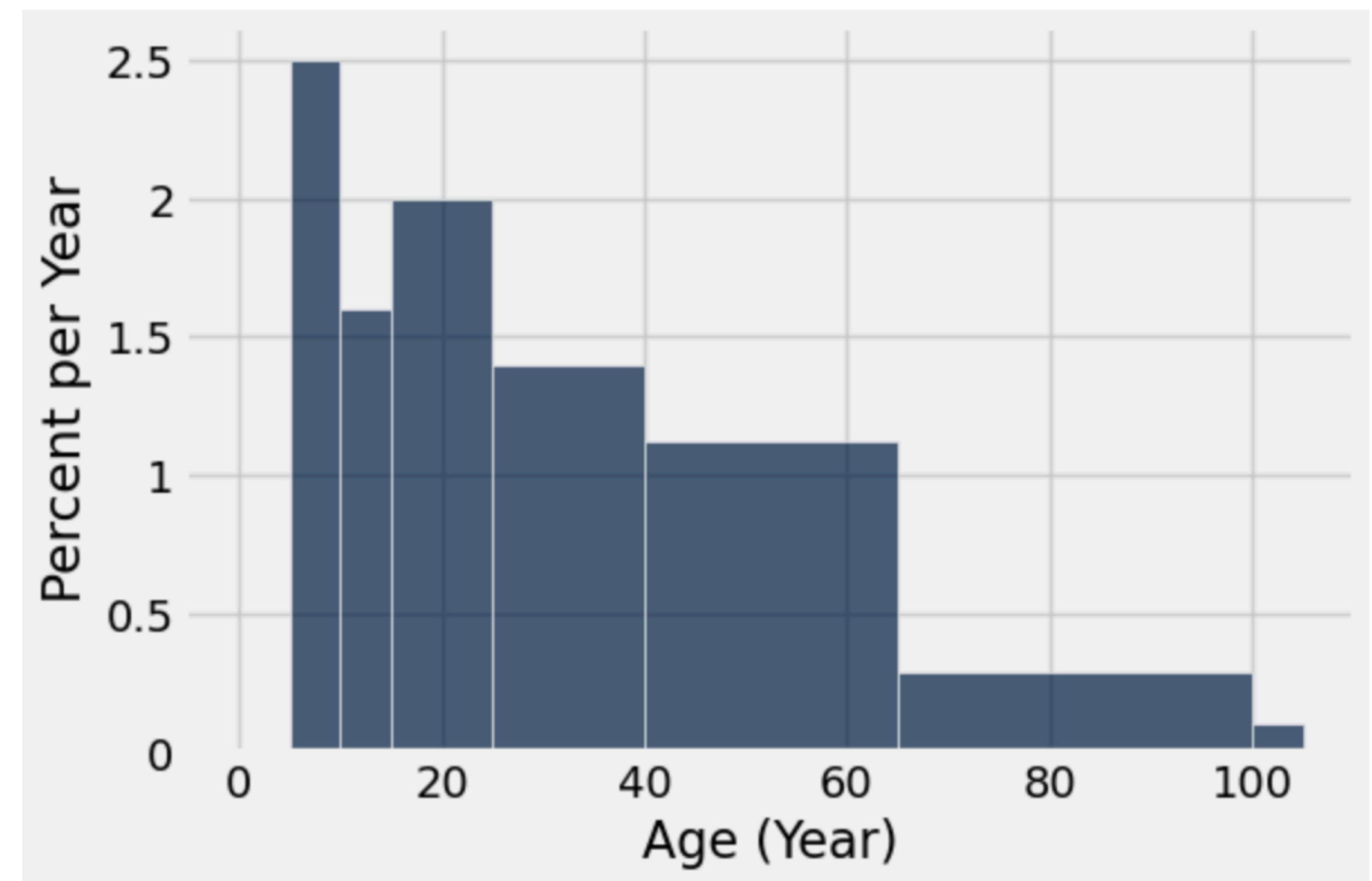
$$\begin{aligned} \text{height} &= \frac{\% \text{ in bin}}{\text{width of bin}} = \frac{29.5 \%}{25 \text{ years}} \\ &= 1.18 \% \text{ per year} \end{aligned}$$

Area Measures Percent

area of bar = percent of entries in bin
= (height of bar) $\times$ (width of bin)

When to use area vs height?

- “How many individuals in the bin?”
 - Use area
- “How crowded is the bin?”
 - Use height



Data Visualization Recap

Why Data Visualizations?

- Charts can convey a lot of information in an interpretable manner
 - People are good at noticing patterns in visual media!
- What we've covered so far:
 - How to generate visualizations
 - Isolating parts of a table that you want to visualize
- What's next:
 - Manipulating the data you have into a format you wish to visualize

Visualizing a Distribution

Bar Chart

- Distribution of **categorical** variable
- Bars have arbitrary (but equal) widths and spaces; in any order
- **Length (or height)** and **area** of bars are proportional to the percent of individuals

Histogram

- Distribution of **numerical** variable
- Horizontal axis is numerical: drawn to scale, no gaps, bins can be unequal
- **Area** of bars is proportional of percent of individuals; **height** measures density

Summary: Charts in this Class

Type	Syntax	Description
Line graph	<code>.plot(x_axis, y_axis)</code>	Sequential numerical data
Scatter Plot	<code>.scatter(x_axis, y_axis)</code>	Non-sequential numerical data
Bar Chart	<code>.barh(column_label)</code>	Distribution of a categorical variable <i>or</i> relationship between a categorical variable and a numerical values
Histogram	<code>.hist(column_label, unit, bins)</code>	Distribution of numerical data

Functions

From Lecture 2: Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)
 - Multiple arguments are separated by a comma



```
abs(-1)
```

1

```
max(4, 200, 7)
```

200

From Lecture 2: Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)
 - Some useful functions aren’t built in, so we tell Python where to find them using **import**

```
import numpy  
numpy.sqrt(4)
```

```
2.0
```

Function Documentation

`sum?`

Signature: `sum(iterable, /, start=0)`

Docstring:

Return the sum of a 'start' value (default: 0) plus an iterable of numbers

When the iterable is empty, return the start value.

This function is intended specifically for use with numeric values and may reject non-numeric types.

Type: `builtin_function_or_method`

Function Documentation

np.where?

Call signature: np.where(*args, **kwargs)

Type: _ArrayFunctionDispatcher

String form: <built-in function where>

Docstring:

where(condition, [x, y], /)

Return elements chosen from `x` or `y` depending on `condition`.

.. note::

When only `condition` is provided, this function is a shorthand for ``np.asarray(condition).nonzero()``. Using `nonzero` directly should be preferred, as it behaves correctly for subclasses. The rest of this documentation covers only the case where all three arguments are provided.

Parameters

condition : array_like, bool

Where True, yield `x`, otherwise yield `y`.

x, y : array_like

Values from which to choose. `x`, `y` and `condition` need to be broadcastable to some shape.

Returns

out : ndarray

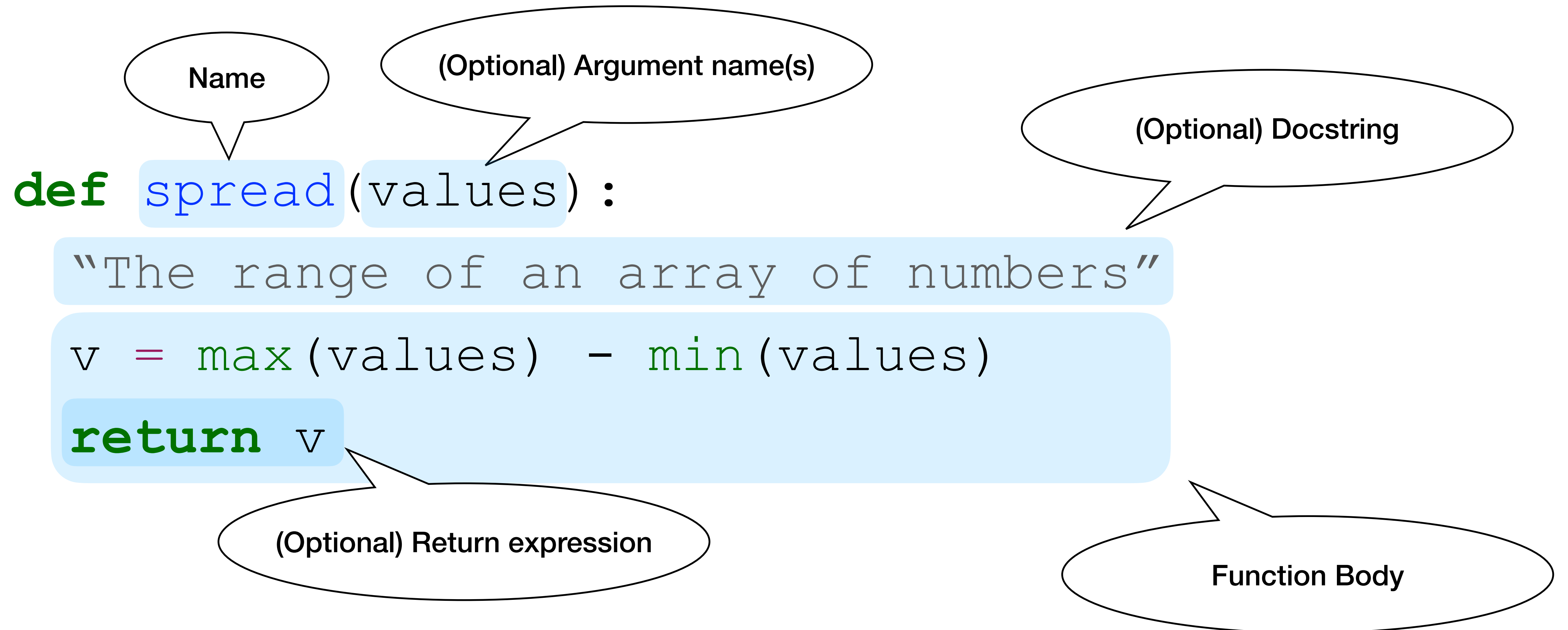
An array with elements from `x` where `condition` is True, and elements from `y` elsewhere.

Defining Functions

- A function gives a name to some code so that you can easily execute the same lines of code many times
- Great for organizing code
- Avoids repeating code
- Can make code easier to read and understand

Anatomy of a Function

Use **def** to define your own function!



Recipe for Defining Functions

It's often easiest to start with an example before defining a function:

- First, think of some examples inputs. What is the code for producing the correct output for these example inputs?
- Try putting the code for your examples into a def statement and return the output
- Try calling the function on different example inputs. Does it give the correct output?
 - If not, walk through your code and see what goes wrong

DEMO

More Tips for Functions

- Avoid naming your function something that already exists
- **return** will immediately exit a function
 - Typically goes at the end
- Later when you write more complicated functions, it's useful to test both that it does what you expect and that it *doesn't* do what you don't expect

```
def is_alice(name):  
    return name=="alice"  
    print("I've gone unnoticed!")
```

```
is_alice("alice")
```

True

```
is_alice("bob")
```

False

Example

What does this function do?

- What type of input do you expect it takes?
- What type of output will it give?
- What's a reasonable name for the function?

```
def f(s):  
    return s / sum(s) * 100
```

Example

What does this function do?

- What type of input do you expect it takes? *Array*
- What type of output will it give? *Array*
- What's a reasonable name for the function?

```
def f(s):  
    return s / sum(s) * 100
```

Example

What does this function do?

- What type of input do you expect it takes? *Array*
- What type of output will it give? *Array*
- What's a reasonable name for the function? *Anything related to percent*

```
def percent(s):  
    return s / sum(s) * 100
```

apply

apply

1. Calls a function on each element in the input column(s)
2. Produces an array containing the output of the function on each input column element

```
tbl.apply(function_name, column_label)
```

```
tbl.apply(function_name, column_label1, column_label2)
```

DEMO

Next Class

- Today
 - Visualizing numerical distributions (Histograms)
 - Functions
- Wednesday
 - Groups